# π-Former: Succinct Proofs of Transformer Inference via a SNARK-Friendly Architecture

Hideaki Takahashi
*Columbia University*
ht2673@columbia.edu

## Abstract

We present π-Former, a succinct argument system for verifiable transformer inference whose central design choice is to make the *model* SNARK-friendly rather than to make the proof system absorb expensive operations. Three architectural changes drive the design: (i) softmax attention is replaced by a kernel attention whose feature map $\phi$ is parametrised as an additively decomposed lookup table, so the only non-arithmetic operation in a block is a structured Lasso lookup; (ii) every projection matrix is constrained to ternary entries $\{-1, 0, +1\}$ scaled by a single learnable $\alpha$ per layer, which removes *all* field multiplications from sumcheck-time weight contractions; and (iii) the lookup tables themselves are *learnable*, so model expressivity is recovered through training rather than through a larger circuit — a free substitution under Lasso, whose prover/verifier cost depends only on table size and query count, not on table contents.

On top of this architecture we build a transparent SNARK over BN254 with the Hyrax polynomial commitment scheme. The block uses *sandwich normalisation* — five LayerNorms per block ($LN_1$, QK-norm on $\mathbf{Q}$ and $\mathbf{K}$, post-attention LN before the residual, $LN_2$) — to keep linear-attention activations and lookup inputs in range, and the prover handles all $5L+1$ LNs in a *single model-level batched LN proof*. The proof system batches all per-block sub-protocols into six cross-block sumchecks (QKV, output projection, attention out, attention context, FFN-Y, FFN-M), one global Lasso proof per lookup family, one shared range-multiplicity commitment per width bucket across the entire model, an in-circuit floor-division proof for the row-normaliser $Z$, and ten cross-cutting Hyrax accumulators that finalise into one MSM pair per group. Residual connections cost no proof at all — they are checked by homomorphic addition of Hyrax commitments. We give cross-block / cross-LN batching soundness theorems and an end-to-end soundness reduction to discrete log on BN254 G1.

## 1 Introduction

Large language models are increasingly deployed as black-box services: the user submits a prompt, a remote operator runs a proprietary model, and the user has no cryptographic guarantee that the returned output came from the advertised model. Verifiable inference [1, 10, 12, 15] addresses this gap through a succinct non-interactive argument: the operator commits to model weights once, then for every input produces a small proof that the verifier can check without re-executing the model and without learning the weights.

The core difficulty is that a transformer block is built from exactly the operations that finite-field arithmetic circuits find expensive. Softmax requires exponentials and a row-wise division; GeLU is transcendental; LayerNorm contains a square root and a division by a running variance; and a single dense projection costs one field multiplication per weight entry. Prior work tackles these on the *proof-system* side: zkLLM [15] introduces `tlookup`, a parallelised lookup argument tailored to softmax and SwiGLU; zkGPT [12] pairs Plonky2 with constraint fusion and circuit squeezing to drive down the cost of approximating softmax. Both keep the model architecture fixed and pay an "unfriendly-operation tax" on every transformer block.

**Our approach.** π-Former starts from a different premise: we change the *model* so that the proof system has nothing expensive to absorb. The design follows three principles, chosen in this order:

1. *SNARK-friendly architecture.* Replace every operation that has no compact arithmetic representation. In particular, replace softmax attention by a kernel attention whose only non-arithmetic component is a structured table lookup, and replace LayerNorm's division and square root by an integer reformulation provable through sumcheck and chunked range proofs.

2. *Ternary weights for free contractions.* Constrain every projection to ternary entries $\{-1, 0, +1\}$ scaled by a per-

layer scalar $\alpha$. A degree-2 sumcheck against a ternary weight matrix materialises only the cheap row $\alpha \cdot \mathbf{W}(\cdot, r)$ and folds the equality polynomial against the ternary entries with $+=/-=$/skip — no field multiplications appear in the contraction [11].

3. *Recover accuracy through learnable lookup, not larger circuits.* The element-wise non-linearity $\phi$ is itself *learnable*: it is parametrised as a sum of small sub-tables, all entries of which are trained end-to-end. Under Lasso [14], the prover and verifier cost depends on the table size and the query count but is otherwise *independent of the table contents*. Letting training adapt the table values therefore restores expressivity at zero proving cost.

The first principle eliminates the worst offenders; the second turns linear contractions into addition-only loops; the third gives the model the freedom to recover accuracy through training rather than through a larger circuit. Together they reduce a transformer block to operations that map directly to a low-degree sumcheck claim or to a Lasso lookup over a small structured table.

**Cross-block batching.** On top of the SNARK-friendly architecture we build a proof system engineered around a single observation: *the same protocol is run once per block, with independent inputs but identical structure*. $\pi$-Former runs six *model-level* batched sumchecks (one per protocol type) that fold all $L$ per-block claims via a Fiat–Shamir random linear combination in an independent challenge $\eta$; the inner sumcheck transcript is shared across all blocks. The same idea applies to Lasso (one global multi-instance proof per lookup family), to range checks (one shared multiplicity commitment per block, plus one for the final layer), to residuals (homomorphic addition of Hyrax commitments; no proof at all), and to the final stage of opening (ten cross-cutting deferred Hyrax accumulators, each finalising in one MSM pair). Each batching step is a standard random-linear-combination argument whose error is dominated by the inner sumcheck error.

**Contributions.**

1. A SNARK-friendly transformer block (§3) in which every operation — attention, normalisation, projection — has a sub-linear verifier circuit, by combining learnable lookup attention, ternary projections, and a constraint-fused LayerNorm.
2. A complete end-to-end SNARK protocol (§4) that batches all $L$ per-block sub-proofs into six model-level cross-block sumchecks, two global Lasso proofs, and ten deferred Hyrax accumulators.
3. Cross-block batching soundness theorems (§5, Theorem 3) and an end-to-end soundness bound (Theorem 8).
4. A complete prototype implemented in $\sim$19 k lines of Rust over BN254 with a matched PyTorch training pipeline whose integer forward pass is bit-identical to the Rust circuit (§6).

**Threat model.** We target the standard non-interactive argument-of-knowledge setting in the random-oracle model. The verifier holds a verifying key $\mathsf{vk}$ binding the model weights and lookup tables, a public input $\mathbf{x}_{\text{in}}$, and a claimed output $\mathbf{y}$. The prover is computationally bounded; its goal is to convince the verifier that $\mathcal{M}_\theta(\mathbf{x}_{\text{in}}) = \mathbf{y}$ for the model committed in $\mathsf{vk}$. Soundness is computational under the discrete-log assumption on BN254 G1, the standard assumption for the Hyrax PCS [18]. The current implementation is not zero-knowledge: weight commitments are public VK entries and intermediate sumcheck evaluations are revealed. We use "verifiable inference" to refer to integrity, not witness privacy, and outline the standard masking extension in §7.

## 2 Background

### 2.1 Notation

We work over the BN254 scalar field $\mathbb{F} = \mathbb{F}_r$ with $r \approx 2^{254}$, and instantiate the Hyrax PCS on the BN254 G1 group $\mathbb{G}$ of the same prime order. We write $L$ for the number of transformer blocks, $T$ for the sequence length, $d$ for the embedding dimension, and $d_{ff}$ for the FFN width; the prototype is single-head ($d_h = d$). A multilinear polynomial $\widetilde{f}$ on $n$ variables is identified with its $2^n$ evaluations on $\{0,1\}^n$; its evaluation at $r \in \mathbb{F}^n$ is denoted $\widetilde{f}(r)$. The equality polynomial $\widetilde{\mathrm{eq}}(r,x) = \prod_i (r_i x_i + (1-r_i)(1-x_i))$ satisfies $\sum_x \widetilde{\mathrm{eq}}(r,x)\widetilde{f}(x) = \widetilde{f}(r)$. Capital bold $\mathbf{X}, \mathbf{W}, \ldots$ denote matrices represented as multilinear polynomials over $\mathbb{F}$ after fixed-point quantisation (§2.6).

### 2.2 The transformer block

A standard pre-norm transformer block [17] computes

$$\mathbf{X}_{\text{mid}} = \mathbf{X}_{\text{in}} + \mathrm{Att}\big(\mathrm{LN}(\mathbf{X}_{\text{in}})\big),$$
$$\mathbf{X}_{\text{out}} = \mathbf{X}_{\text{mid}} + \mathrm{FFN}\big(\mathrm{LN}(\mathbf{X}_{\text{mid}})\big),$$

where standard self-attention is $\mathrm{Att}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \mathrm{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d})\mathbf{V}$ and LayerNorm [3] is $\mathrm{LN}(\mathbf{x}) = \gamma \odot (\mathbf{x}-\mu)/\sqrt{\sigma^2+\varepsilon} + \beta$. Each of softmax, GeLU and LayerNorm contains operations (exp, division, square root) without compact polynomial representations, so they dominate SNARK complexity in prior work.

### 2.3 Sumcheck

For an MLE $f : \mathbb{F}^n \to \mathbb{F}$ of individual degree $\delta$, the sumcheck protocol [13, 16] reduces a claim $H = \sum_{x \in \{0,1\}^n} f(x)$ to a single evaluation $f(r)$ at random $r \in \mathbb{F}^n$ in $n$ rounds. Soundness error is $\le n\delta/|\mathbb{F}|$ and verifier cost is $O(n\delta)$ field operations.

π-Former uses the degree-2 variant for products of two MLEs, the cubic variant for LayerNorm variance, and a multi-batched variant $\sum_k \lambda_k \sum_x f_k(x) g_k(x)$ for cross-block batching.

### 2.4 Hyrax PCS

For a multilinear polynomial of $2^n = 2^{\nu+\sigma}$ evaluations arranged as a $2^\nu \times 2^\sigma$ matrix $M$, Hyrax commits to each row as a vector Pedersen commitment $C_i = \mathrm{MSM}(\mathbf{g}, M[i])$ over public "nothing-up-my-sleeve" generators $g_j = \mathrm{SHA3}(\texttt{"piformer-hyrax-gen"} \| j) \cdot G$, and opens at $r = (r_L \| r_R)$ by sending the row-collapsed vector $w'_j = \sum_i L_i(r_L) M[i][j]$, where $L_i$ are multilinear Lagrange bases. The verifier checks (i) a homomorphic commitment identity $\sum_i L_i(r_L) C_i = \mathrm{MSM}(\mathbf{g}, w')$ and (ii) an inner product identity $\langle R(r_R), w' \rangle = v$ [18]. Hyrax is transparent (no trusted setup), computationally binding under DL on $\mathbb{G}$, and has $O(\sqrt{N})$ commitments, openings, and proof size.

### 2.5 Lasso

Given a sub-table $T \in \mathbb{F}^{2^m}$ and queries $\{(\mathrm{idx}_j, v_j)\}_{j=1}^N$ with $v_j = T[\mathrm{chunk}(\mathrm{idx}_j)]$, Lasso [2, 5, 14] proves all queries are consistent with $T$ via a single batched sumcheck for $\sum_x \widetilde{T}(x) L(x) = \sum_j \rho^j T[\mathrm{ch}_j]$ where $L(x) = \sum_j \rho^j \widetilde{\mathrm{eq}}(\mathrm{bin}(\mathrm{ch}_j), x)$ and $\rho$ is Fiat–Shamir-derived. A composite table that decomposes *additively* into $c$ sub-tables (the canonical Lasso "structured" condition) is committed as $c$ small tables of size $2^{B/c}$ rather than one monolithic table of size $2^B$. Critically for our design, the prover/verifier cost depends on the table size and the query count, but is otherwise *independent of the table contents*: changing the values of $T$ does not change the cost of proving lookups against $T$.

### 2.6 Fixed-point encoding

Real-valued tensors are quantised to integers and embedded in $\mathbb{F}$ by $x_\mathbb{F} = \lfloor x/s \rfloor \bmod r$, with the scale $s$ chosen so that no intermediate value overflows $r$ (e.g., for $B$-bit activations and $k$-bit weights, $T \cdot d_h \cdot 2^B \cdot 2^k \ll r$). The PyTorch training pipeline applies the *same* integer arithmetic in its forward pass, so the exported witness exactly matches the field arithmetic in the Rust circuit, eliminating the train–prove gap.

## 3 The π-Former Architecture

A π-Former block (Figure 1) is a sandwich-normalised linear-attention block. Let $\mathbf{X}_{\mathrm{in}} \in \mathbb{F}^{T \times d}$ denote the input hidden state; the block computes

$$\mathbf{X}_{\mathrm{n1}} = \mathrm{LN}_1(\mathbf{X}_{\mathrm{in}}), \tag{1}$$

$$\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathbf{X}_{\mathrm{n1}} \mathbf{W}_{Q,K,V} \ \text{(ternary)}, \tag{2}$$

$$\mathbf{Q}^{\mathrm{n}} = \mathrm{LN}_Q(\mathbf{Q}), \ \ \mathbf{K}^{\mathrm{n}} = \mathrm{LN}_K(\mathbf{K}) \ \text{(QK-norm)}, \tag{3}$$

$$\mathbf{N} = \phi(\mathbf{Q}^{\mathrm{n}}) \big(\phi(\mathbf{K}^{\mathrm{n}})^\top \mathbf{V}\big) \ \text{(LLA numerator)}, \tag{4}$$

$$Z_t = \phi(\mathbf{q}_t^{\mathrm{n}})^\top \textstyle\sum_s \phi(\mathbf{k}_s^{\mathrm{n}}) \ \text{(per-row denominator)}, \tag{5}$$

$$\mathbf{Y}^{\mathrm{att}}_{tj} = \lfloor S \cdot \mathbf{N}_{tj} / Z_t \rfloor \ \text{(fixed-point row norm)}, \tag{6}$$

$$\mathbf{O}_{\mathrm{att}} = \mathbf{Y}^{\mathrm{att}} \mathbf{W}_O + \mathbf{b}_O \ \text{(ternary)}, \tag{7}$$

$$\mathbf{X}_{\mathrm{mid}} = \mathbf{X}_{\mathrm{in}} + \mathrm{LN}_{\mathrm{aon}}(\mathbf{O}_{\mathrm{att}}) \ \text{(sandwich norm)}, \tag{8}$$

$$\mathbf{X}_{\mathrm{n2}} = \mathrm{LN}_2(\mathbf{X}_{\mathrm{mid}}), \tag{9}$$

$$\mathbf{O}_{\mathrm{ffn}} = \phi(\mathbf{X}_{\mathrm{n2}} \mathbf{W}_1) \mathbf{W}_2 \ \text{(structured-lookup FFN)}, \tag{10}$$

$$\mathbf{X}_{\mathrm{out}} = \mathbf{X}_{\mathrm{mid}} + \mathbf{O}_{\mathrm{ffn}}. \tag{11}$$

After $L$ such blocks the model applies one final LayerNorm and a ternary language-model head, so the model contains $5L+1$ LayerNorms in total. The constant $S = \mathrm{ATTN\_NORM\_SCALE} = 64$ re-introduces fractional precision before the floor division.

The two LayerNorms that are not present in a textbook pre-norm transformer are essential for our integer pipeline:

- *QK-norm* ($\mathrm{LN}_Q, \mathrm{LN}_K$) pins the dynamic range of $\mathbf{Q}, \mathbf{K}$ so the kernel features $\phi(\mathbf{Q}^{\mathrm{n}}), \phi(\mathbf{K}^{\mathrm{n}})$ and the denominator $Z$ stay inside the lookup-table range regardless of the single-$\alpha$ ternary projection scale.
- *Sandwich norm* ($\mathrm{LN}_{\mathrm{aon}}$) absorbs the unbounded magnitude of linear attention before the residual sum. Softmax attention is row-stochastic and naturally bounded; the linear numerator $\phi(\mathbf{Q})(\phi(\mathbf{K})^\top \mathbf{V})$ has no such bound, and without normalisation the residual stream drifts in magnitude and destabilises downstream LayerNorms.

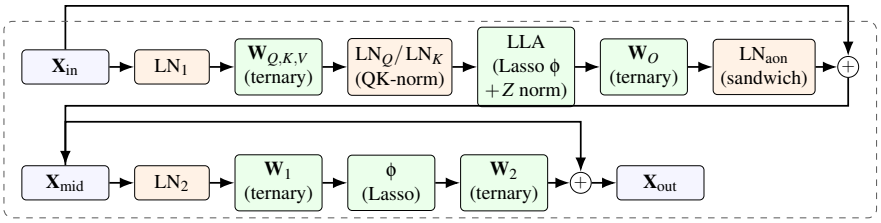
Both are LayerNorms over the same primitive (§3.4); the only SNARK-side cost is more LN witnesses, which the model-level batched LN proof of §4 folds into a single transcript and the bucketed range batch (§3.5) folds into a shared multiplicity commitment. The SNARK now also proves the row-normalisation step $\mathbf{Y}^{\mathrm{att}} = \lfloor S \cdot \mathbf{N} / Z \rfloor$ end-to-end via the attention-normalisation sub-protocol of §3.1.

The three architectural ingredients are described below.

### 3.1 Learnable Lookup Attention (LLA)

Standard softmax attention requires $T^2$ exponentials and $T$ row-wise divisions. We replace the kernel with a positive-definite inner product $\kappa(\mathbf{q}, \mathbf{k}) = \langle \phi(\mathbf{q}), \phi(\mathbf{k}) \rangle$ on the post-QK-norm tensors $\mathbf{Q}^{\mathrm{n}}, \mathbf{K}^{\mathrm{n}}$, where $\phi : \mathbb{R}^{d_h} \to \mathbb{R}^{d_h}$ is applied element-wise and realised as a structured lookup (§3.2). The block computes the un-normalised numerator $\mathbf{N}$ and the per-row denominator $Z$ as

$$\mathbf{N}_{tj} = \phi(\mathbf{q}_t^{\mathrm{n}})^\top \big(\textstyle\sum_s \phi(\mathbf{k}_s^{\mathrm{n}}) \mathbf{v}_s^\top\big)_j, \ \ Z_t = \phi(\mathbf{q}_t^{\mathrm{n}})^\top \textstyle\sum_s \phi(\mathbf{k}_s^{\mathrm{n}}). \tag{12}$$




**Cross-block batching.** The five LNs per block ($\text{LN}_1$, QK-norm $\text{LN}_Q/\text{LN}_K$, sandwich $\text{LN}_{\text{aon}}$, $\text{LN}_2$), plus the final LN, are folded into one *model-level* batched LN proof. Ternary projections fold into `qkv`/`oproj`/`ffn` sumchecks; LLA inner products fold into `batch_attn_out`/`ctx`; the row-normaliser $Z$ is enforced by a cubic multi-batched sumcheck with range proofs on the floor-division residuals (§3.1); Lasso $\phi$ folds into one global multi-instance proof per lookup family; range checks fold into one shared multiplicity commitment per width bucket; residuals are homomorphic Hyrax addition.

Figure 1: A single π-Former block. Boxes are matrices; rounded green nodes are operations the SNARK proves; orange nodes are LayerNorms. *Sandwich norm* ($\text{LN}_{\text{aon}}$) is applied to the attention output before the residual; *QK-norm* ($\text{LN}_Q$, $\text{LN}_K$) is applied to $\mathbf{Q},\mathbf{K}$ after the ternary projection and before $\phi$. Ternary projections contain no field multiplications in the contraction; Lasso lookup cost is independent of table contents; residual $+$ is checked by homomorphic addition of Hyrax commitments and requires no proof.

The context matrix $\mathbf{M} = \sum_s \phi(\mathbf{k}_s^{\text{n}})\mathbf{v}_s^\top \in \mathbb{F}^{d_h \times d_h}$ depends only on the keys/values and is computed once: per-token output cost is $O(d_h^2)$ and the per-block prover cost becomes $O(Td_h^2)$ instead of $O(T^2 d_h)$ [7].

**Floor-division proof for row normalisation.** Unlike the previous prototype, π-Former now proves the row normalisation $\mathbf{Y}^{\text{att}}_{tj} = \lfloor S{\cdot}\mathbf{N}_{tj}/Z_t \rfloor$ end-to-end. The witness includes auxiliary tensors $\mathbf{N}, \mathbf{Y}^{\text{att}}, Z, \text{rem}, \text{diff}$ where $\text{rem}_{tj} = S\mathbf{N}_{tj} - Z_t\mathbf{Y}^{\text{att}}_{tj}$ and $\text{diff}_{tj} = Z_t - 1 - \text{rem}_{tj}$, all committed via Hyrax. At a single random point $r_{\text{norm}} \in \mathbb{F}^{t_{\text{bits}}+d_{\text{bits}}}$ the prover runs one cubic multi-batched sumcheck enforcing

$$S{\cdot}\mathbf{N} \;-\; \text{rem} \;-\; Z{\cdot}\mathbf{Y}^{\text{att}} \;=\; 0, \tag{13}$$

$$\lambda\big(Z \;-\; 1 \;-\; \text{rem} \;-\; \text{diff}\big) \;=\; 0, \tag{14}$$

merged into one cubic claim by the Fiat–Shamir scalar $\lambda$. Range proofs on $\text{rem}, \text{diff}$ (added to the 64-bit width bucket of §3.5) prove $\text{rem}, \text{diff} \geq 0$, which combined with the second identity gives $0 \leq \text{rem} < Z$ — the floor-division witness. A second sumcheck binds $Z_t$ to $\phi(\mathbf{q}_t^{\text{n}}){\cdot}\sum_s \phi(\mathbf{k}_s^{\text{n}})$ via the already-committed $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$ MLEs; a causal variant binds $Z$ to the prefix sum.

**Lookup binding to post-QK-norm tensors.** The Lasso queries for $\phi$ are evaluated on the post-QK-norm tensors, not on the pre-norm $\mathbf{Q},\mathbf{K}$. The prover commits both $C_{\mathbf{Q}}, C_{\mathbf{K}}$ (used by the QKV projection sumcheck) *and* $C_{\mathbf{Q}^{\text{n}}}, C_{\mathbf{K}^{\text{n}}}$ (used as the $\phi$ inputs); the model-level $\phi(\mathbf{Q}), \phi(\mathbf{K})$ Lasso binds its query indices to $C_{\mathbf{Q}^{\text{n}}}, C_{\mathbf{K}^{\text{n}}}$ via one shared Hyrax batch open, and the QKV cross-block sumcheck binds $C_{\mathbf{Q}}, C_{\mathbf{K}}$ separately. The LayerNorm sub-proof for $\text{LN}_Q/\text{LN}_K$ (folded into the model-level batched LN proof) ties the two together.

## 3.2 Additive sub-table feature map

Rather than committing to a fixed kernel, π-Former parametrises $\phi$ as a sum of *learnable* sub-table lookups. Let $B$ be the input bit-width and $c \mid B$ the decomposition depth, $m = B/c$ bits per chunk. For scalar $x$, define the integer index $x_{\text{int}} = \text{clamp}(\lfloor x/s \rfloor, 0, 2^B - 1)$ and chunk extractor $\chi_i(x_{\text{int}}) = (x_{\text{int}} \gg im)\,\&\,(2^m - 1)$. Then

$$\phi(x) \;=\; \sum_{i=0}^{c-1} T_i\big[\chi_i(x_{\text{int}})\big], \tag{15}$$

where $T_0, \ldots, T_{c-1} \in \mathbb{R}^{2^m}$ are learnable sub-tables, initialised so $\sum_i T_i \approx \text{GELU}$ and trained end-to-end with the straight-through estimator [4]. For $B = 16, c = 2$ this replaces a 65,536-entry monolithic table with two 256-entry tables: *exactly* the Lasso structured condition.

The crucial property exploited by π-Former is that, under Lasso (§2.5), the cost of proving a batch of lookups against $T_i$ depends only on the size of $T_i$ and on the number of queries; it does not depend on the values of $T_i$. The model can therefore train the entries of every $T_i$ from scratch without incurring any additional proving cost. We implement `StructuredLookupActivation` on the Python side with a sub-table forward pass and an STE backward pass; the integer indices emitted by the Python forward are bit-identical to those proven on the Rust side.

## 3.3 Ternary projections

All projection matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O, \mathbf{W}_1, \mathbf{W}_2$ and the LM head are constrained to entries in $\mathcal{T} = \{-1, 0, +1\}$ scaled by a single learnable scalar $\alpha \in \mathbb{F}$ per layer, following BitNet b1.58 [11]. The forward pass is $\mathbf{y} = \alpha(\mathbf{x}\mathbf{W}) + \mathbf{b}$ with

$\mathbf{W} \in \mathcal{T}^{m\times n}$. Training uses the magnitude-thresholded mapping $w \mapsto \text{sign}(w)\cdot[|w| > 0.7\overline{|w|}]$ with STE; the scalar $\alpha$ is multiplied in *after* the matrix product so the in-circuit weight stays ternary.

In the SNARK, a degree-2 sumcheck for a ternary contraction

$$\mathbf{Y}(r_t, r_d) - \mathbf{b}(r_d) = \alpha \sum_k \mathbf{X}(r_t, k)\mathbf{W}(k, r_d)$$

materialises only the cheap vector $\alpha \cdot \mathbf{W}(\cdot, r_d)$, never a field weight matrix; the prover folds the equality polynomial against ternary entries with $+=/-=$/skip rather than field multiplications. The Rust prover represents weights as `Vec<Vec<TernaryValue>>` with `TernaryValue` $\in$ `{ONE, ZERO, MINUSONE}` and folds via a specialised `eval_cols_ternary` helper. A self-contained Lasso check (`attention/ternary_check.rs`) over the 4-element table $[0, 1, r-1, 0]$ enforces ternarity by an integer encoding $\text{enc}(w) \in \{0,1,2\}$; this module is currently a stand-alone protocol and is a candidate to fold into setup-time preprocessing so the verifying key carries a binding proof that $\mathbf{W} \in \mathcal{T}^{m\times n}$.

## 3.4 Constraint-fused LayerNorm

Let $\mathbf{x} \in \mathbb{Z}^d$ be a row, $\text{sum\_x} = \sum_j x_j$, $\text{var\_x} = \sum_j (d\cdot x_j - \text{sum\_x})^2$ the integer variance numerator, and $\sigma = \lfloor \sqrt{\text{var\_x}}/d \rfloor$. $\pi$-Former proves the entire LayerNorm at a single random row index $r_t \in \mathbb{F}^{t_{\text{bits}}}$ in four steps, with no field divisions and no square roots:

1. **Mean.** Degree-2 sumcheck for $\widetilde{\text{sum\_x}}(r_t) = \sum_j x_{\text{col}}(j)$ where $x_{\text{col}} = \widetilde{\mathbf{x}}.\text{fix\_row}(r_t)$.
2. **Variance.** Cubic sumcheck for $\widetilde{\text{var\_x}}(r_t) = \sum_j h(j)^2$ with $h(j) = d\cdot x_{\text{col}}(j) - \widetilde{\text{sum\_x}}(r_t)$.
3. **$\sigma$ floor-sqrt.** Show $\sigma$ is the integer floor square-root by range-checking $\text{var\_x} - (d\sigma)^2 \geq 0$ and $(d\sigma + d)^2 - 1 - \text{var\_x} \geq 0$ through the chunked Lasso range proof of §3.5.
4. **Y constraint fusion.** For a single random $(r_{y_t}, r_{y_d})$, audit $\gamma \odot (d\mathbf{x} - \text{sum\_x}) + \beta \cdot d\sigma \approx d\sigma \cdot \mathbf{y}$ (up to integer rounding) by another range proof on the lo/hi residuals; the $\gamma\mathbf{X}$ and $\sigma\mathbf{Y}$ legs are fused into a single multi-batched cubic sumcheck.

The verifier evaluates $\gamma$ and $\beta$ directly from the VK in $O(d)$ field operations per layer; everything else is one shared opening per witness.

**Model-level batched LN proof.** The end-to-end prover does not emit one independent LN sub-proof per LN. Instead, after Phase 1 has absorbed all $5L+1$ LN IO commitments ($\text{LN}_1$, $\text{LN}_Q$, $\text{LN}_K$, $\text{LN}_{\text{aon}}$, $\text{LN}_2$ per block, plus the final LN), one call to ProveLNsBatched folds the mean, variance, $\sigma$-floor, and Y-fusion sub-claims of every LN into a single batched cubic sumcheck under Fiat–Shamir powers of an independent challenge. Soundness follows from the same Schwartz–Zippel argument as cross-block batching (Theorem 3) with the role of $L$ played by $5L+1$.

## 3.5 Globally batched range proof

The range proof shows $V[i] \in [0, 2^{\text{bits}})$ via a chunked Lasso approach over identity tables, with CHUNK_BITS $= 16$. The implementation supports two width buckets, bits $\in \{32, 64\}$, and routes each witness to the narrowest bucket that contains it. The LayerNorm $\sigma/y$ witnesses use 64-bit range proofs (LAYERNORM_RANGE_BITS $= 64$); the optional attention- normalisation residuals rem, diff (§3.1) are also 64-bit. Each non-empty bucket is processed independently and produces one $m_{\text{com}}$.

For $B$ witnesses in a single bucket with $C = \lceil \text{bits}/16 \rceil$ chunks each (so $C \in \{2, 4\}$), Phase 1 commits all $V_0^{(b)}, \ldots, V_{C-1}^{(b)}$ via Hyrax and merges all chunk arrays into one global multiplicity vector $m[v] = \sum_{b,c,i} [\text{chunk}^{(b,c)}[i] = v]$ committed once. Phase 2 runs one degree-2 sumcheck per witness binding $V^{(b)}$ to a random $r_v^{(b)}$ and checks $V^{(b)}(r_v^{(b)}) = \sum_c 2^{16c} V_c^{(b)}(r_v^{(b)})$ through one Hyrax batch open per witness. Phase 3 opens $m(r_m)$ once and checks the LogUp identity [6] against the identity table $T_{\text{id}}[i] = i$. The transcript-ordering invariant (Theorem 5) is that $m_{\text{com}}$ is absorbed before any Phase 2 sumcheck challenge is sampled.

The model-level prover collects *all* range witnesses across the entire model into a single bucket-routed list: $10L+2$ LN witnesses ($\sigma, y$ for the five LNs per block plus the final LN's two), and $2L$ extra witnesses (rem, diff per block) when normalised attention is enabled. Sharing one $m_{\text{com}}$ per bucket therefore saves $(B-1) \times \sqrt{2^{16}}$ G1 MSMs per bucket, and the savings scale with $L$ rather than being capped at the per-block level.

# 4 The $\pi$-Former Protocol

This section gives the protocol. Algorithm 1 describes preprocessing; Algorithm 2 the model-level prover; Algorithm 3 the per-block "Phase 1" that commits intermediate matrices and proves both LayerNorms; Algorithm 4 the cross-block batched sumcheck primitive; and Algorithm 6 the verifier. The LayerNorm sub-prover (Algorithm 5) and the globally batched range proof are described in §3.4 and §3.5.

### 4.1 Preprocessing

**Algorithm 1** $\pi$-Former. $\mathsf{Setup}(\theta;\text{params})$

**Input:** model weights $\theta = \{\mathbf{W}_Q^\ell, \mathbf{W}_K^\ell, \mathbf{W}_V^\ell, \mathbf{W}_O^\ell, \mathbf{W}_1^\ell, \mathbf{W}_2^\ell, \gamma^\ell, \beta^\ell\}_{\ell=1}^L$, final-LN params, LM head $\mathbf{W}_h$; params $(T, d, d_{ff}, d_h, B, c)$.
**Output:** proving key pk, verifying key vk.

derive Hyrax generators $g_j \leftarrow \mathrm{SHA3}(\texttt{"piformer-hyrax-gen"} \| j) \cdot G$.
**for** each weight matrix $\mathbf{W} \in \theta \cup \{\mathbf{W}_h\}$ **do**
  $C_{\mathbf{W}} \leftarrow \mathsf{Com}(\mathbf{W})$ ▷ Hyrax commit to ternary entries
**end for**
**for** each lookup sub-table $T_0, \ldots, T_{c-1}$ **do**
  $C_{T_i} \leftarrow \mathsf{Com}(T_i)$
**end for**
$\mathsf{vk} \leftarrow \{C_{\mathbf{W}}\} \cup \{C_{T_i}\} \cup \{\gamma^\ell, \beta^\ell\} \cup \{g_j\}$
$\mathsf{pk} \leftarrow \mathsf{vk} \cup \{\text{raw ternary arrays, sub-tables}\}$
**return** $(\mathsf{pk}, \mathsf{vk})$

The verifying key contains *no* raw weights, only commitments. The proving key additionally carries the ternary $\{-1, 0, +1\}$ arrays and the lookup sub-tables so the prover can fold the equality polynomial against them in $O(Td)$ time without any field multiplications.

### 4.2 The model-level prover

**Algorithm 2** $\pi$-Former. $\mathsf{Prove}(\mathsf{pk}, W, \mathsf{inst})$

**Input:** proving key pk, witness $W$, instance inst (Lasso queries for $\phi(\mathbf{Q}), \phi(\mathbf{K}), \phi(\mathbf{M})$ at every block).
**Output:** model proof $\pi$.

$\mathsf{FS.init}(\texttt{"piformer"})$; $C_{\mathbf{x}_{\text{in}}} \leftarrow \mathsf{Com}(\mathbf{x}_{\text{in}})$; $\mathsf{FS.absorb}(C_{\mathbf{x}_{\text{in}}})$
**Phase 1: per-block commit + LN IO absorption**
$C_{\text{cur}} \leftarrow C_{\mathbf{x}_{\text{in}}}$
**for** $\ell = 1, \ldots, L$ **do**
  $p_\ell \leftarrow \textsc{Phase1}(W_\ell, C_{\text{cur}}, \mathsf{pk}_\ell, \mathsf{FS})$ ▷ commits 5 LN IO matrices, $\phi$ inputs, optional attn-norm aux
  $C_{\mathbf{X}_{\text{mid}}^\ell} \leftarrow C_{\text{cur}} + p_\ell.C_{\text{LN}_{\text{aon}}.y}$ ▷ sandwich-norm residual
  $C_{\text{cur}} \leftarrow C_{\mathbf{X}_{\text{mid}}^\ell} + p_\ell.C_{\mathbf{O}_{\text{ffn}}}$
**end for**
**Phase 2: global random point**
$r_{td} = (r_t \| r_{\text{out}}) \leftarrow \mathsf{FS.challenge}(t_{\text{bits}} + d_{\text{bits}})$
**Phase 3: four cross-block sumchecks**
$\pi_{qkv} \leftarrow \textsc{CrossBatch}(\{f_\ell^{qkv}, g_\ell^{qkv}\}_\ell, \mathsf{FS})$
$\pi_{op} \leftarrow \textsc{CrossBatch}(\{f_\ell^{op}, g_\ell^{op}\}_\ell, \mathsf{FS})$
$\pi_{ao} \leftarrow \textsc{CrossBatch}(\{f_\ell^{ao}, g_\ell^{ao}\}_\ell, \mathsf{FS})$
$\pi_{ac} \leftarrow \textsc{CrossBatch}(\{f_\ell^{ac}, g_\ell^{ac}\}_\ell, \mathsf{FS})$
**Phase 4: attention normalisation sumcheck (when normalised mode)**
$r_{\text{norm}} \leftarrow \mathsf{FS.challenge}(t_{\text{bits}} + d_{\text{bits}})$; $\lambda \leftarrow \mathsf{FS.challenge}$
$\pi_{\text{atn}} \leftarrow \textsc{ProveCubicMulti}\big(\{\mathbf{N}_\ell, Z_\ell, \mathbf{Y}_\ell^{\text{att}}, \text{rem}_\ell, \text{diff}_\ell\}_\ell, \lambda, \mathsf{FS}\big)$
$\pi_Z \leftarrow \textsc{CrossBatch}(\{\phi(\mathbf{Q}^{\text{n}})_\ell, \sum_s \phi(\mathbf{K}^{\text{n}})_\ell\}_\ell, \mathsf{FS})$
**Phase 5: global FFN Lasso + FFN-Y/M sumchecks**
commit $\{\mathbf{A}_\ell, \mathbf{M}_\ell\}_\ell$; $\pi_{\text{lasso}}^{\text{ffn}} \leftarrow \mathsf{Prove\text{-}LassoMulti}\big(\{\mathsf{inst}_\ell.\phi(\mathbf{M})\}_\ell, \{(C_{\mathbf{A}_\ell}, \mathbf{A}_\ell)\}_\ell, \mathsf{FS}\big)$
bind FFN indices to $\{C_{\mathbf{M}_\ell}\}_\ell$ via one Hyrax batch open
$\pi_{ffy} \leftarrow \textsc{CrossBatch}(\{f_\ell^{ffy}, g_\ell^{ffy}\}_\ell, \mathsf{FS})$
$\pi_{ffm} \leftarrow \textsc{CrossBatch}(\{f_\ell^{ffm}, g_\ell^{ffm}\}_\ell, \mathsf{FS})$
**Phase 6: bucketed range batch + model-level batched LN proof**
**for** bits $\in \{32, 64\}$ **do**
  $\pi_{\text{rng,bits}} \leftarrow \textsc{ProveRangeBatched}(\text{all witnesses in bucket, bits, F}$
**end for**
$\pi_{\text{ln}} \leftarrow \textsc{ProveLNsBatched}(\text{all } 5L{+}1 \text{ LNs}, \mathsf{FS})$
**Phase 7: LM head + $\mu$-challenges + batch opens**
$\pi_{\text{head}} \leftarrow \mathsf{Prove\text{-}Proj}(W.\text{LM-head}, \mathsf{FS})$
drain 5 deferred-accumulator $\mu$-challenges
$\pi_{\text{inter}} \leftarrow \mathsf{Open\text{-}Batch}\big(\{\mathbf{Q}_\ell, \mathbf{K}_\ell, \mathbf{V}_\ell, \mathbf{O}_{\text{att}}^\ell, \mathbf{O}_{\text{ffn}}^\ell\}_{\ell=1}^L, r_{td}\big)$
$\{\pi_{\text{open}}^{(j)}\} \leftarrow \mathsf{Open\text{-}Batch}(\{\mathbf{X}_{\text{n1}}^\ell, \mathbf{W}_Q^\ell, \ldots, C_{\phi(\mathbf{Q}^{\text{n}})_\ell}, C_{\phi(\mathbf{K}^{\text{n}})_\ell}\}_\ell, r^{(j)})$
(when normalised) $\mathsf{Open\text{-}Batch}([\mathbf{N} \mid \mathbf{Y}^{\text{att}} \mid \text{rem} \mid \text{diff}]_\ell, r_{\text{norm}})$
**Phase 8: global $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$ Lasso**
commit $\{\phi(\mathbf{Q}^{\text{n}})_\ell, \phi(\mathbf{K}^{\text{n}})_\ell\}_\ell$; bind indices to $\{C_{\mathbf{Q}_\ell^{\text{n}}}, C_{\mathbf{K}_\ell^{\text{n}}}\}$
$\pi_{\text{lasso}}^{\text{qk}} \leftarrow \mathsf{Prove\text{-}LassoMulti}\big(\{\mathsf{inst}_\ell.\phi(\mathbf{Q}^{\text{n}}), \mathsf{inst}_\ell.\phi(\mathbf{K}^{\text{n}})\}_\ell, \ldots, \mathsf{FS}\big)$
**return** $\pi$

The key structural choices are: *(i) one global random point $r_{td}$ is drawn after all per-block commitments are absorbed*, so all six cross-block sumchecks can share it; *(ii) ten cross-cutting Hyrax accumulators* (LayerNorm row / $td$, projection W/b, LM-head W/b, range $\sigma$/Y/$m$, intermediate) are pushed by every claim and finalised once at the end of the *verifier*, with one MSM pair per group; *(iii) residuals are verified by linearity of the Hyrax PCS:* $C_{\mathbf{X}_{\text{mid}}} = C_{\mathbf{X}_{\text{in}}} + C_{\mathbf{O}_{\text{att}}}$ is computed by the verifier from the existing commitments and requires no proof.

The two model-level Lasso instances merit separate mention. Each is a *committed-output* Lasso: before sampling the multi-Lasso batching challenges, the prover absorbs the Hyrax commitments to the lookup outputs ($C_{\mathbf{A}_\ell}$ for FFN, $C_{\phi(\mathbf{Q}^{\text{n}})_\ell}$ and $C_{\phi(\mathbf{K}^{\text{n}})_\ell}$ for attention). A second sumcheck inside the Lasso multi-proof binds the combined lookup grand sum to the committed output polynomials at the sumcheck terminal point. Lookup indices are likewise bound to the committed *post-norm* input tensors ($C_{\mathbf{M}_\ell}, C_{\mathbf{Q}^{\text{n}}_\ell}, C_{\mathbf{K}^{\text{n}}_\ell}$) via one shared Hyrax batch opening, so a cheating prover cannot pick query indices independently of the committed inputs and cannot mix pre-norm and post-norm tensors.

## 4.3 Per-block Phase 1

**Algorithm 3** PHASE1$(W_\ell, C_{\text{cur}}, \mathsf{pk}_\ell, \mathsf{FS})$

**Input:** block witness, current input commitment, block PK.
**Output:** intermediate commitments; *no* LN sumcheck or range proof emitted at this stage.

commit pre-/post-QK-norm $\mathbf{Q}, \mathbf{K}, \mathbf{V}$: $C_{\mathbf{Q}}, C_{\mathbf{K}}, C_{\mathbf{V}}, C_{\mathbf{Q}^{\text{n}}}, C_{\mathbf{K}^{\text{n}}}$
commit LN outputs: $C_{\mathbf{X}_{\text{n1}}}, C_{\text{LN}_{\text{aon}}.y}, C_{\mathbf{X}_{\text{n2}}}$
commit O-projection / FFN outputs: $C_{\mathbf{O}_{\text{att}}}, C_{\mathbf{O}_{\text{ffn}}}$
(when normalised attention) commit $C_{\mathbf{N}}, C_{\mathbf{Y}^{\text{att}}}, C_Z, C_{\text{rem}}, C_{\text{diff}}$
absorb $\text{LN}_1$ IO: $\mathsf{FS.absorb}(C_{\text{cur}}, C_{\mathbf{X}_{\text{n1}}})$
absorb $C_{\mathbf{Q}}, C_{\mathbf{K}}, C_{\mathbf{V}}$ and (when normalised) attn-norm aux commits
absorb $\text{LN}_Q$ IO: $\mathsf{FS.absorb}(C_{\mathbf{Q}}, C_{\mathbf{Q}^{\text{n}}})$
absorb $\text{LN}_K$ IO: $\mathsf{FS.absorb}(C_{\mathbf{K}}, C_{\mathbf{K}^{\text{n}}})$
$\mathsf{FS.absorb}(C_{\mathbf{O}_{\text{att}}})$
absorb $\text{LN}_{\text{aon}}$ IO: $\mathsf{FS.absorb}(C_{\mathbf{O}_{\text{att}}}, C_{\text{LN}_{\text{aon}}.y})$
$C_{\mathbf{X}^\ell_{\text{mid}}} \leftarrow C_{\text{cur}} + C_{\text{LN}_{\text{aon}}.y}$ ▷ sandwich-norm residual 1
absorb $\text{LN}_2$ IO: $\mathsf{FS.absorb}(C_{\mathbf{X}^\ell_{\text{mid}}}, C_{\mathbf{X}_{\text{n2}}})$
$\mathsf{FS.absorb}(C_{\mathbf{O}_{\text{ffn}}})$
**return** all commitments

The only truly per-block step is Phase 1: it commits the per-block intermediate matrices and absorbs all five LN IO commitments (plus the optional attention-normalisation aux commits) into the transcript in a fixed order. *No per-block LN sumcheck and no per-block range proof are emitted at this stage*; both are deferred to model-level batched calls (Phase 6 of Algorithm 2). The remaining sub-protocols (QKV, output projection, attention, FFN) are *not* run independently per block either; they are batched cross-block in Phases 3–5 of Algorithm 2. Each per-block proof carries the intermediate Hyrax commitments, the FFN $C_{\mathbf{A}_\ell}, C_{\mathbf{M}_\ell}$, the per-block $C_{\phi(\mathbf{Q}^{\text{n}})_\ell}, C_{\phi(\mathbf{K}^{\text{n}})_\ell}$, the optional attention-normalisation aux commits, and the per-block scalar evaluations consumed by the cross-block batch sumchecks.

## 4.4 Cross-block batched sumcheck

**Algorithm 4** CROSSBATCH$(\{f_\ell, g_\ell, H_\ell\}_{\ell=1}^L, \mathsf{FS})$

**Input:** per-block MLEs $f_\ell, g_\ell : \mathbb{F}^n \to \mathbb{F}$ and per-block targets $H_\ell$.
**Output:** batched proof $\pi_{\text{batch}}$ and shared random point $r \in \mathbb{F}^n$.

**for** $\ell = 1, \ldots, L$ **do** absorb $H_\ell$ and per-block constants into $\mathsf{FS}$
**end for**
$\eta \leftarrow \mathsf{FS.challenge}(\texttt{"batch\_eta"})$
$H^\star \leftarrow \sum_\ell \eta^{\ell-1} H_\ell$; $P(x) \leftarrow \sum_\ell \eta^{\ell-1} f_\ell(x) g_\ell(x)$
**for** $i = 1, \ldots, n$ **do**
  send $s_i(X) = \sum_{x \in \{0,1\}^{n-i}} P_i(X, x)$ at $X \in \{0, 1, 2\}$
  $r_i \leftarrow \mathsf{FS.challenge}$
**end for**
**return** $(\eta, (s_i)_{i=1}^n), r = (r_1, \ldots, r_n)$

The verifier recovers $H^\star$ from the per-block claims and $\eta$, then runs the standard sumcheck verifier on $\pi_{\text{batch}}$ to a final point $r$. From $r$ the verifier checks the terminal claim $P(r) = \sum_\ell \eta^{\ell-1} f_\ell(r) g_\ell(r)$ against the per-block evaluations opened later via the cross-block batch opens. By Theorem 3 this protocol replaces $L$ independent sumcheck transcripts with one of length $n$ at a soundness cost of an additive $(L-1)/|\mathbb{F}|$.

## 4.5 Constraint-fused LayerNorm prover

The pseudocode below describes the LayerNorm protocol *for one LN witness*. The end-to-end prover does not invoke it once per LN; instead, it collects all $5L+1$ LN witnesses (in fixed order: per block $\text{LN}_1, \text{LN}_Q, \text{LN}_K, \text{LN}_{\text{aon}}, \text{LN}_2$; then the final LN) and runs a single PROVELNSBATCHED call as Phase 6 of Algorithm 2. That batched call shares the row challenge $r_t$, the variance / Y-fusion sumchecks, and the deferred Hyrax accumulator pushes across all LN instances, with per-LN claims combined by Fiat–Shamir powers of an independent challenge.

**Algorithm 5** PROVELN$(W, \{C_{\mathbf{x}}, C_{\mathbf{y}}\}, (\mathsf{rps}_{\sigma}, \mathsf{rps}_{y}), \mathsf{FS})$

**Input:** witness $W = \{\mathbf{x}, \mathbf{y}, \text{sum\_x}, \text{sq\_sum\_x}, \sigma\}$, IO commitments, range witnesses (precomputed by Phase 1).

commit $C_{\text{sum\_x}}, C_{\text{sq\_sum}}, C_{\sigma}$; absorb everything into FS
$r_t \leftarrow \mathsf{FS.challenge}(t_{\text{bits}})$
*// 1. Mean sumcheck (degree-2)*
$(\pi^{\text{mean}}, r_d^{\text{m}}) \leftarrow \mathsf{Prove\text{-}Sumcheck}_2(\widetilde{\mathbf{x}}_{\text{col}}, \mathbb{1}, \widetilde{\text{sum\_x}}(r_t), \mathsf{FS})$
*// 2. Variance sumcheck (cubic)*
$h(j) \leftarrow d \cdot \widetilde{\mathbf{x}}_{\text{col}}(j) - \widetilde{\text{sum\_x}}(r_t)$
$(\pi^{\text{var}}, r_d^{\text{v}}) \leftarrow \mathsf{Prove\text{-}Sumcheck}_3(h, h, h, \widetilde{\text{var\_x}}(r_t), \mathsf{FS})$
*// 3. Sigma range proof (consumes* $\mathsf{rps}_{\sigma}$*)*
$r_{\sigma} \leftarrow \mathsf{rps}_{\sigma}.r$; check $\text{var} - (d\sigma)^2 \geq 0$ and $(d\sigma + d)^2 - 1 - \text{var} \geq 0$ at $r_{\sigma}$
*// 4. Y constraint fusion:* $\gamma\mathbf{X} + \sigma\mathbf{Y}$ *legs (multi-cubic)*
$r_y \leftarrow \mathsf{rps}_y.r$; $\alpha \leftarrow \mathsf{FS.challenge}(\texttt{"layernorm\_alpha"})$
$(\pi^{\gamma\sigma}, r_f) \leftarrow \mathsf{Prove\text{-}Sumcheck}_3^{\text{multi}}\big([\gamma \odot \widetilde{\mathbf{x}}_{\text{ry}}, \widetilde{\sigma}_{\text{ry}} \odot \widetilde{\mathbf{y}}_{\text{ry}}], \alpha, \mathsf{FS}\big)$
push 10 Hyrax claims into deferred accumulators $\mathsf{acc}_t, \mathsf{acc}_{td}$
**return** $\{\pi^{\text{mean}}, \pi^{\text{var}}, \pi^{\gamma\sigma}, \text{evaluations}\}$

## 4.6 The verifier

**Algorithm 6** $\pi$-Former. $\mathsf{Verify}(\mathsf{vk}, \pi, \mathsf{inst}, \mathbf{x}_{\text{in}}, \mathbf{y})$

**check** $\mathsf{Com}(\mathbf{x}_{\text{in}}) = \pi.C_{\mathbf{x}_{\text{in}}}$ and $\mathsf{Com}(\mathbf{y}) = \pi.C_{\mathbf{y}}$
$\mathsf{FS.init}(\texttt{"piformer"})$; absorb $\pi.C_{\mathbf{x}_{\text{in}}}$; initialise 10 deferred Hyrax accumulators
$C_{\text{cur}} \leftarrow \pi.C_{\mathbf{x}_{\text{in}}}$
**for** $\ell = 1, \ldots, L$ **do** ▷ Phase 1 lockstep: absorb all 5 LN IO + aux
    absorb $\text{LN}_1$ IO; absorb $C_{\mathbf{Q}}, C_{\mathbf{K}}, C_{\mathbf{V}}$ + (opt.) attn-norm aux
    absorb $\text{LN}_Q$ IO ($C_{\mathbf{Q}}, C_{\mathbf{Q}^{\text{n}}}$), $\text{LN}_K$ IO ($C_{\mathbf{K}}, C_{\mathbf{K}^{\text{n}}}$), $C_{\mathbf{O}_{\text{att}}}$
    absorb $\text{LN}_{\text{aon}}$ IO; $C_{\mathbf{X}_{\text{mid}}} \leftarrow C_{\text{cur}} + p_{\ell}.C_{\text{LN}_{\text{aon}}.y}$
    absorb $\text{LN}_2$ IO; $C_{\text{cur}} \leftarrow C_{\mathbf{X}_{\text{mid}}} + p_{\ell}.C_{\mathbf{O}_{\text{ffn}}}$
**end for**
$r_{td} \leftarrow \mathsf{FS.challenge}(t_{\text{bits}} + d_{\text{bits}})$
**verify** four cross-block sumchecks $\pi_{qkv}, \pi_{op}, \pi_{ao}, \pi_{ac}$
(when normalised) **verify** $\pi_{\text{atn}}$ at $r_{\text{norm}}$ and $\pi_Z$
**verify** global FFN Lasso, then $\pi_{ffy}, \pi_{ffm}$
**verify** bucketed range batch (one $m_{\text{com}}$ per width)
**verify** model-level batched LN proof (all $5L+1$ LNs)
**verify** LM-head proof and logits opening
drain 5 deferred-accumulator $\mu$-challenges
**verify** global intermediate batch open at $r_{td}$
**verify** cross-block matrix-type batch opens
(when normalised) **verify** batch opens at $r_{\text{norm}}$
FINALISE(10 deferred accumulators) ▷ one MSM pair per group
**verify** global $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$ Lasso
**return** ACCEPT

# 5 Soundness

We sketch the soundness analysis; the full proofs follow standard Schwartz–Zippel and PCS-binding arguments.

**Lemma 1** (Sumcheck soundness)**.** *For an MLE $f$ of individual degree $\leq \delta$, a cheating prover can make the sumcheck for $H = \sum_{x \in \{0,1\}^n} f(x)$ accept a false claim $H' \neq H$ with probability at most $n\delta/|\mathbb{F}|$.*

**Lemma 2** (Hyrax binding)**.** *Under the discrete-log assumption on $\mathbb{G}$, the Hyrax PCS is computationally binding: no PPT adversary can open a single commitment to two different evaluations at the same point with non-negligible advantage.*

**Theorem 3** (Cross-block batching is sound)**.** *Let* CROSSBATCH *(Algorithm 4) reduce $L$ per-block claims $\{H_\ell\}$ to one batched claim $H^\star = \sum_\ell \eta^{\ell-1} H_\ell$, and let the inner sumcheck be sound with error $\leq n\delta/|\mathbb{F}|$. Then* CROSSBATCH *is sound with total error*

$$\varepsilon_{\text{batch}} \;\leq\; \frac{L-1}{|\mathbb{F}|} + \frac{n\delta}{|\mathbb{F}|}.$$

*Proof.* The challenge $\eta$ is drawn from FS *after* every per-block claim and per-block witness commitment is absorbed. The prover is therefore committed to $\{H_\ell\}$ before learning $\eta$. If any single claim is false, $\hat{H}_{\ell^\star} \neq H_{\ell^\star}$, then $P(\eta) = \sum_\ell \eta^{\ell-1}(\hat{H}_\ell - H_\ell)$ is a non-zero polynomial in $\eta$ of degree $\leq L-1$. By Schwartz–Zippel, $P(\eta) = 0$ holds for at most $L-1$ values of $\eta$, so a uniformly random $\eta$ catches the false claim with probability $\geq 1 - (L-1)/|\mathbb{F}|$. Conditional on the combined claim being honest, the inner sumcheck has soundness $n\delta/|\mathbb{F}|$. A union bound gives the stated error. □

**Theorem 4** (Homomorphic residuals)**.** *Let $C_{\mathbf{A}} = \mathsf{Com}(\mathbf{A})$, $C_{\mathbf{B}} = \mathsf{Com}(\mathbf{B})$. The verifier-computed commitment $C_{\mathbf{A}} + C_{\mathbf{B}}$ binds $\mathbf{A} + \mathbf{B}$ unconditionally given Hyrax binding (Lemma 2).*

*Proof.* Hyrax commits row-by-row as MSMs over public generators $\mathbf{g}$. By bilinearity of MSM, $C_{\mathbf{A}} + C_{\mathbf{B}} = (\text{MSM}(\mathbf{g}, A_i) + \text{MSM}(\mathbf{g}, B_i))_i = (\text{MSM}(\mathbf{g}, A_i + B_i))_i = \mathsf{Com}(\mathbf{A} + \mathbf{B})$. Binding of $\mathbf{A} + \mathbf{B}$ then follows from binding of $\mathbf{A}$ and $\mathbf{B}$. □

**Theorem 5** (Globally batched range proof is sound)**.** *Sharing one $m_{\text{com}}$ per width bucket across all range witnesses in the bucket (LN $\sigma$/$y$ across all blocks plus the final LN, plus the optional attention-normalisation residuals) is sound: a cheating prover cannot supply chunk values outside $[0, 2^{16})$ for any single witness.*

*Proof.* Each bucket is processed independently and the argument applies per bucket. The transcript-ordering invariant is that $m_{\text{com}}$ is absorbed before any per-witness sumcheck challenge $r_v^{(b)}$ in the bucket is sampled. Since $m$ aggregates all chunk occurrences across all witnesses and all $C = \lceil \text{bits}/16 \rceil$ chunks per witness, the prover must commit

to $m$ before seeing any sumcheck challenge. If the prover supplied a chunk value $v^* \geq 2^{16}$, the LogUp identity [6] would require $m[v^*] \cdot T_{\text{id}}[v^*]$ to balance, but $T_{\text{id}}$ has no entry for $v^*$, contradicting the committed $m$ (Lemma 2). □

**Theorem 6** (Model-level batched LayerNorm is sound)**.** *The single* ProveLNsBatched *call covering all $5L+1$ LNs in the model (per block: $\text{LN}_1, \text{LN}_Q, \text{LN}_K, \text{LN}_{\text{aon}}, \text{LN}_2$; plus the final LN) is as sound as $5L+1$ independent LN sub-proofs run with independent challenges.*

*Proof.* The batched LN proof folds the $5L+1$ per-LN claims via Fiat–Shamir powers of an independent batching challenge $\eta$, drawn after every per-LN IO commitment $(\mathbf{x}, \mathbf{y}, \text{sum\_x}, \text{sq\_sum}, \sigma)$ has been absorbed into the transcript and after the bucketed range batch has committed to all $\sigma/y$ chunks. The same Schwartz–Zippel argument as Theorem 3 applies (with $L$ replaced by $5L+1$): a single false LN claim makes the combined polynomial in $\eta$ non-zero of degree $\leq 5L$, caught with probability $\geq 1 - 5L/|\mathbb{F}|$. The shared inner sumchecks (mean, variance, Y-fusion) inherit Lemma 1 soundness on the combined claim. □

**Theorem 7** (Attention-normalisation sumcheck is sound)**.** *When normalised attention is enabled, the cubic multi-batched sumcheck for $S \cdot \mathbf{N} - \text{rem} - Z \cdot \mathbf{Y}^{\text{att}} = 0$ and $\lambda(Z - 1 - \text{rem} - \text{diff}) = 0$, combined with the range proofs on* rem, diff *(Theorem 5) and the auxiliary $Z$ sumcheck, soundly enforces the floor-division identity $\mathbf{Y}^{\text{att}}_{tj} = \lfloor S \cdot \mathbf{N}_{tj} / Z_t \rfloor$ at every $(t, j)$.*

*Proof sketch.* Range proofs constrain $\text{rem} \geq 0$ and $\text{diff} \geq 0$. The second identity then gives $\text{rem} \leq Z - 1$, i.e. $0 \leq \text{rem} < Z$. The first identity expresses $Z \cdot \mathbf{Y}^{\text{att}} = S \cdot \mathbf{N} - \text{rem}$ with the unique quotient given $0 \leq \text{rem} < Z$. Both identities are checked at one Fiat–Shamir random point of $t_{\text{bits}} + d_{\text{bits}}$ variables with cubic-sumcheck error and a $\lambda$-batching error of $O(1)/|\mathbb{F}|$. The auxiliary $Z$ sumcheck binds $Z_t$ to $\phi(\mathbf{q}^{\text{n}}_t) \cdot \sum_s \phi(\mathbf{k}^{\text{n}}_s)$ via the already-committed $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$ MLEs. Hyrax binding closes the loop. □

**Theorem 8** ($\pi$-Former end-to-end soundness)**.** *For any PPT adversary $\mathcal{A}$, the probability that* $\mathsf{Verify}(\mathsf{vk}, \pi, \mathsf{inst}, \mathbf{x}_{\text{in}}, \mathbf{y}) = \mathsf{ACCEPT}$ *but $\mathcal{M}_\theta(\mathbf{x}_{\text{in}}) \neq \mathbf{y}$ is at most*

$$\varepsilon_{\text{sound}} \leq \frac{m_{\text{sc}}\, n_{\max} \delta_{\max} + m_{\text{batch}}(L-1) + m_{\text{open}}}{|\mathbb{F}|} + \varepsilon_{\text{DL}},$$

*where $m_{\text{sc}}$ is the number of sumcheck invocations after batching, $m_{\text{batch}} = 6$ is the number of cross-block batches, $m_{\text{open}}$ covers the random-linear-combination checks used in batched openings and lookups, $n_{\max}$ is the maximum number of sumcheck variables, $\delta_{\max} \leq 3$ is the maximum round-polynomial degree, and $\varepsilon_{\text{DL}}$ is the DL advantage of $\mathcal{A}$ on $\mathbb{G}$.*

*Proof sketch.* By hybrid argument over the $L$ blocks. (i) By Lemma 2, any forged Hyrax opening yields a DL solution. (ii) Block-level chaining is sound by Theorem 4, with the sandwich-norm residual $C_{\mathbf{X}_{\text{mid}}} = C_{\mathbf{x}_{\text{in}}} + C_{\text{LN}_{\text{aon}}.y}$ inheriting binding from $C_{\text{LN}_{\text{aon}}.y}$. (iii) Within each block, the bucketed global range batch over all witnesses is sound by Theorem 5; the model-level batched LN proof covering all $5L+1$ LNs is sound by Theorem 6; when normalised attention is enabled, the floor-division identity is sound by Theorem 7; the cross-block sumchecks are sound by Theorem 3; the committed-output Lasso is sound by Lemma 2 plus Lemma 1. The Lasso index-binding step ties the lookup queries to the post-QK-norm tensors $C_{\mathbf{Q}^{\text{n}}}, C_{\mathbf{K}^{\text{n}}}$, which are themselves bound to $C_{\mathbf{Q}}, C_{\mathbf{K}}$ through the LN-batched proof for $\text{LN}_Q, \text{LN}_K$. (iv) The final layer plus LM head reduce to one LN plus one projection. A union bound over all sub-protocols gives the bound. For typical parameters ($L \leq 12$, $n_{\max} \leq 64$, $\delta_{\max} \leq 3$) over BN254, the statistical term is below $2^{-240}$ and is dominated by the DL term. □

Table 1: $\pi$-Former Rust implementation, lines of code per module.

| Module | LoC |
|---|---|
| `poly/` (dense MLE, equality, helpers) | ∼0.6 k |
| `pcs/` (Hyrax PCS, batch accumulator) | ∼1.3 k |
| `subprotocols/` (sumcheck ×3, GKR combine) | ∼1.5 k |
| `lookup/` (Lasso multi, range batched, quant.) | ∼3.4 k |
| `attention/` (LN, projection, LLA, ternary check) | ∼4.6 k |
| `ffn/` (FFN sub-prover) | ∼0.7 k |
| `cross_layer/` (cross-layer projection sumcheck) | ∼0.6 k |
| `prover.rs`, `verifier.rs`, `setup.rs` | ∼6.0 k |
| `bin/piformer/` (CLI, codecs, sample) | ∼0.5 k |
| **Total** | ∼19 k |

## 5.1 Zero knowledge

The current implementation is *not* zero-knowledge: weight commitments are public VK entries, and intermediate evaluation claims (e.g., $\mathbf{X}_{\text{n1}}(r_t, r_d)$) are revealed in the sumcheck transcripts. Standard techniques apply: blinding all witness polynomials with random masking terms before commitment, and using zero-knowledge variants of sumcheck and Hyrax. We discuss this further in §7.

# 6 Implementation

We implement $\pi$-Former in ∼19 k lines of Rust over BN254 plus a PyTorch training pipeline. The Rust prover/verifier and CLI are organised as in Table 1.

**PyTorch pipeline.** The training pipeline implements `TernaryLinear` (with STE), `StructuredLookupActivation` ($\phi$ with sub-table forward plus STE backward), and `LinearAttentionLayer`. The integer forward pass used to construct the witness

is bit-identical to the Rust circuit, so a witness exported from PyTorch is accepted by the Rust prover with no quantisation gap. The exporter also emits the Lasso instances ($\phi(\mathbf{Q}), \phi(\mathbf{K}), \phi(\mathbf{M})$ tables, query indices, and outputs) directly from the same integer arithmetic.

**Single-head, no causal mask, no ZK.** The current Rust prover supports single-head attention with no causal masking and no zero-knowledge masking layer; multi-head splits the attention sumchecks into parallel batches with the same protocol structure, and ZK is an additional blinding layer over committed witnesses (§7).

## 7 Discussion and Limitations

**What is and is not in the SNARK.** The Rust prover proves the full normalised LLA output $\mathbf{Y}^{\text{att}} = \lfloor S \cdot \phi(\mathbf{Q}^{\text{n}})(\phi(\mathbf{K}^{\text{n}})^{\top}\mathbf{V})/Z \rfloor$ followed by the ternary output projection $\mathbf{O}_{\text{att}} = \mathbf{Y}^{\text{att}}\mathbf{W}_O + \mathbf{b}_O$ and the sandwich-norm $\text{LN}_{\text{aon}}(\mathbf{O}_{\text{att}})$ before the residual sum. The Python pipeline supports two attention modes: `normalized_fixed` (proven end-to-end by the Rust circuit, including the floor division) and `prover` (un-normalised numerator only, retained for legacy export). `normalized_float` is rejected at export time. The current implementation lacks causal masking (an experimental causal $Z$ sumcheck exists in the code but is not enabled in the production export), multi-head attention, and zero-knowledge masking. None of these changes the protocol architecture: causal masking is a sumcheck-time indicator polynomial, multi-head splits the attention sumchecks into parallel batches, and ZK is an additional blinding layer over committed witnesses.

**Model-quality evaluation.** Because π-Former changes the attention kernel and constrains weights to ternary values, a deployment-quality comparison must train matched baselines under the same parameter budget, context length, dataset, and quantisation regime, then report both task quality and proof cost. We leave this to future work and present π-Former here as evidence that the co-designed architecture is efficiently provable.

**From prototype to deployment.** Our reference implementation is CPU-only and single-threaded. GPU MSM and parallel sumcheck [12] would translate directly to π-Former: every dominant inner loop (cross-block batched sumcheck, Lasso multi-instance, batched Hyrax open) is data-parallel, and the global Hyrax accumulators are pure MSMs at the boundary. Streaming token generation is the natural fit for Nova-style folding [9]: the running context matrix $\mathbf{M} = \phi(\mathbf{K})^{\top}\mathbf{V}$ is the IVC accumulator, and one folding step per token yields constant amortised proving cost.

**Beyond π-Former.** The Rust codebase already contains two stand-alone sub-protocols that are not yet wired into the end-to-end prover. The first is an in-circuit ternary-weight check (`attention/ternary_check.rs`) via a 4-element Lasso table $[0, 1, r-1, 0]$; folding this into setup yields a verifying key that carries a binding proof $\mathbf{W} \in \mathcal{T}^{m \times n}$. The second is a single cubic sumcheck (`cross_layer/projection.rs`) that collapses $L$ per-layer projection sumchecks into one of length $\log L + \log d_{\text{in}}$ by introducing a layer-index variable; this gives a further factor of $L$ saving on the projection transcript. Together with an EVM-targeted Hyrax verifier and a zero-knowledge masking layer, these lay out the path from a research prototype to an on-chain verifiable inference service.

## 8 Related Work

**Verifiable ML.** zkCNN [10] gave the first GKR-based proof system for neural network inference, focused on convolutional networks. [1] extends ZK to proofs of *training* for deep networks. zkLLM [15] adapts a custom IOP to softmax LLM inference using `tlookup`-style structured lookups, with `zkAttn` as a tailored attention sub-protocol; zkGPT [12] uses a Plonky2 backend with constraint fusion across operations of a transformer block. Both target the standard softmax + GeLU + LayerNorm stack and pay a significant cost for every non-arithmetic operation. π-Former departs by co-designing the model and proof system: every operation is either a sumcheck or a Lasso lookup by construction, and the model is allowed to recover expressivity through learnable lookup tables that are free under Lasso.

**Sumcheck and lookup arguments.** The sumcheck protocol underlies modern transparent SNARKs [13, 16]. Lasso provides a practical lookup argument with sub-linear prover cost on structured-decomposable tables [2, 5, 14], and is the natural fit for the additive sub-table structure of π-Former's $\phi$. LogUp-style multiplicity arguments [6] back the globally batched range proof. Hyrax [18] gives a transparent, discrete-log-based polynomial commitment with $O(\sqrt{N})$ proofs.

**Linear attention and ternary networks.** Linear attention [7] replaces softmax with a feature-map kernel and recovers the $O(Td^2)$-vs-$O(T^2d)$ complexity trade-off; [19] projects the keys and values to a low-rank space; Reformer [8] uses LSH for sparsity. BitNet b1.58 [11] introduced ternary weights as a replacement for FP16 dense layers in LLMs, motivated by inference energy on edge devices; π-Former re-purposes the same primitive for SNARK proving cost.

## 9 Conclusion

We presented π-Former, a SNARK for verifiable transformer inference in which the model is made SNARK-friendly so the

proof system has nothing expensive to absorb. Three architectural changes drive the design: softmax becomes a kernel attention with a learnable additive-lookup feature map, every projection becomes ternary so its sumcheck contraction has no field multiplications, and LayerNorm becomes an integer reformulation provable through sumcheck and chunked range proofs. The proof system batches per-block sub-protocols into six model-level sumchecks, two global Lasso proofs, and one global range proof per block, with all final Hyrax openings flowing through ten deferred batch accumulators. We implemented the system in $\sim$19 k lines of Rust over BN254 and gave formal cross-block batching soundness theorems. We leave a deployment-quality model evaluation and the engineering steps (multi-head, ZK masking, GPU MSM, on-chain verifier) to future work; we believe $\pi$-Former provides a credible foundation for production-grade verifiable transformer inference.

## References


[1] Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. Zero-knowledge proofs of training for deep neural networks. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4316–4330, 2024.

[2] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: Snarks for virtual machines via lookups. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33. Springer, 2024.

[3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[4] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[5] Oleg Fomenko and Anton Levochko. Understanding lasso: A novel lookup argument protocol. *Cryptology ePrint Archive*, 2025.

[6] Ulrich Haböck. Improving logarithmic derivative lookups using gkr. *Cryptology ePrint Archive*, 2022.

[7] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

[8] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

[9] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*, pages 359–388. Springer, 2022.

[10] Tianyi Liu, Xiang Xie, and Yupeng Zhang. Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2968–2985, 2021.

[11] Shuming Ma, Hongyu Wang, Shaohan Huang, Xingxing Zhang, Ying Hu, Ting Song, Yan Xia, and Furu Wei. Bitnet b1. 58 2b4t technical report. *arXiv preprint arXiv:2504.12285*, 2025.

[12] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. {zkGPT}: An efficient non-interactive zero-knowledge proof framework for {LLM} inference. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2045–2063, 2025.

[13] Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In *Annual International Cryptology Conference*, pages 704–737. Springer, 2020.

[14] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with lasso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 180–209. Springer, 2024.

[15] Haochen Sun, Jason Li, and Hongyang Zhang. zkllm: Zero knowledge proofs for large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4405–4419, 2024.

[16] Justin Thaler. *Proofs, arguments, and zero-knowledge*, volume 4. Foundations and Trends in Privacy and Security, 2022.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[18] Riad S Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zksnarks without trusted setup. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 926–943. IEEE, 2018.

[19] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.